ztensor: Anonymous Incentive Subnets from Scope-Linkable Ring Signatures, with One Signer and Verifiers on Six Execution Environments

The ztensor authors*

October 2026

Abstract

In incentive subnets of the Bittensor kind, miners produce work, validators score it, and a consensus rule turns scores into emission. Both roles act under persistent public identities, so a validator can favour or punish a known miner and a miner can identify, and retaliate against, the validator that scored it down. We present *ztensor*, a protocol that keeps the rules of such a subnet while removing the identities behind every action. Miners and validators form two public sets of keys. Every action is a scope-linkable ring signature (LSAG) whose key image enforces exactly one action per member per scope. A miner's image in an epoch scope is its pseudonymous *tag*. A validator's image in a per-tag scope is a *nullifier*. A claim re-creates the miner's tag and so can only be made by its owner. We pair this with *Zuma consensus*, a variant of Yuma consensus for validators without identities: every validator weighs one, and each score is clipped to the highest score that half of the validator set supports. We prove that a coalition smaller than half the set cannot raise a tag's consensus above what the honest validators support. One signer produces every signature, and six independently written verifiers check them byte for byte: Cairo on Starknet, Rust/wasm on NEAR, Solidity/Yul on Ethereum, sBPF on Solana, an ink! contract on Substrate's pallet-contracts, and an off-chain wasm contract with post-quantum (ML-DSA) receipts. Verification cost is linear in the ring size on every chain we fitted ($R^2 \geq 0.9999$). On Substrate it is $6.4 + 5.71\,n$ ms of reference time, so a 64-member ring uses 29% of the per-extrinsic limit. Priced at the mainnet fees and token prices of 2026-10-20, a ring-4 score costs between USD 0.0019 (Starknet, estimated) and USD 3.52 (Ethereum L1); on Astar, a public pallet-contracts chain, it costs USD 0.0061. Per epoch, the cost grows with miners times validators. We show that the usual compressions, batched weights and multi-epoch claims, link actions that the protocol keeps apart. Secret committees drawn from the validators' own nullifiers, followed by settling each epoch with one validity proof, keep every action unlinkable and are modeled to cut a Bittensor-sized subnet's daily cost by a factor of 307 to 5776. Committees cost security, not privacy, and we quantify that cost. We also report what the construction does not give: anonymity is bounded by the ring and by network metadata, it does not survive a quantum adversary, and an earlier ring-selection rule leaked claimants through ring intersection. We describe that leak and the fix we adopted.

1 Introduction

Bittensor [24] organises machine-learning work into *subnets*. In each subnet, miners serve a task and validators score the miners; Yuma consensus [21] then aggregates the validators' stake-weighted

*Source, data and the scripts that generate every number, table and figure in this paper: `orbit/ztensor` (`paper/collect.py`, `paper/figures.py`). Measurement snapshot: 2026-10-20.

scores into emission. Every participant is a public hotkey, and so every score is attributable. Two problems follow.

1. **Favouritism.** A validator that knows which miner it is scoring can favour an affiliated miner or punish a competitor. Its score depends on the miner's identity, not on the work.

2. **Retaliation and collusion markets.** A miner that knows who scored it down can retaliate or negotiate. A miner and a validator that can recognise each other can contract on scores.

Both problems rely on the parties knowing *who* they are dealing with. Neither needs anything else. Every rule a subnet enforces can be stated without identities: miners are registered; each miner submits once per epoch; each validator scores each submission at most once; payouts follow a consensus rule; and each payout is claimed once, by the miner that earned it. This paper shows that all of these rules can be enforced on chain, by contracts that never learn which member acted.

Approach. We replace identity with *membership plus a scope-bound pseudonym*. A linkable ring signature [15] proves that "some member of this public ring signed". Its key image $I = x \cdot H_{\mathrm{p}}(\sigma)$ is deterministic in the signer's secret x and a *scope* σ. Two signatures by the same key in the same scope therefore share an image, while images in different scopes cannot be linked. Choosing the scopes defines the protocol: (epoch) for miners, (epoch, tag) for validators, and the miner's epoch scope again for claims.

Contributions.

- **Protocol** (Section 4). Four actions, *submit*, *score*, *settle* and *claim*, built only from scope choices on one LSAG instance over the STARK curve with Poseidon [12] challenges. We state the security properties the subnet needs and reduce each to a standard property of the signature scheme (Section 4.1).

- **Zuma consensus** (Section 5). A version of Yuma consensus for anonymous validators. We prove a clipping bound for coalitions below the quorum, and show that settling a tag on chain costs at most 101 storage reads.

- **One signer, six verifiers** (Section 6). A single Cairo prover and its bit-identical Python port sign for every chain. The verifiers are written natively for Starknet, NEAR, Ethereum, Solana, Substrate and an off-chain post-quantum ledger, and each test suite pins its verifier to signatures made by the Cairo prover. The Substrate verifier is new in this paper (Section 6.1).

- **Measurements** (Section 7). Linear cost models per chain, fitted from deterministic resource accounting; the per-transaction limit that sets each chain's maximum ring; and mainnet prices per action.

- **Cost at scale, privacy kept** (Section 8). A per-epoch cost model whose dominant term grows with miners $\times$ validators, and two ladders of cuts. The common cuts (batched weights, lazy claims) link actions. Secret committees drawn from the nullifiers, followed by private settlement in one validity proof, keep every signature unlinkable and reduce a Bittensor-sized subnet's daily cost 307–5776-fold (modeled).

- **Limitations, stated precisely** (Section 9). This includes a ring-intersection leak in our earlier ring-selection rule and its fix.

Non-goals. ztensor does not hide *that* someone is a subnet member, how many members there are, or the content of submitted work. It does not provide Sybil resistance [6], which stays with whoever admits keys to the sets. It does not move tokens: payouts are credits in contract storage, so the system cannot be used as a general-purpose mixer.

2 Background and related work

Ring signatures and linkability. Ring signatures [25] let a signer hide among an ad hoc set of public keys. Linkable ring signatures [15] add a tag that reveals when two signatures share a signer. The *key image* of CryptoNote [29] and Monero's RingCT [20] is the same idea, used there to prevent double spends. Traceable ring signatures [10] make the tag depend on an application-chosen issue, which is what our *scope* is. Bender, Katz and Morselli [4] give a hierarchy of anonymity and unforgeability definitions, which we use informally. Empirical work on Monero [14, 18] shows how ring anonymity erodes in deployment, through decoy selection, temporal analysis and ring intersection. Our Section 9 is an instance of the last of these.

Nullifiers. Zerocash [2] prevents double spending with a nullifier derived from a secret and revealed when spending; it proves membership with a SNARK. Our validator nullifier plays the same role with a ring signature instead of a succinct proof. That trades proof size linear in the ring for a transparent, setup-free scheme whose verifier is a few hundred lines on each chain.

Bittensor and Yuma. In Yuma consensus [21], each validator's weights are clipped to a stake-weighted κ-majority consensus, and validators earn dividends through exponentially smoothed bonds. Bonds and dividends both need a validator identity that persists across rounds, which is exactly what ztensor removes (Section 5).

Execution environments. The Starknet STARK curve and Poseidon are native to Cairo [3, 11]. NEAR [27], Substrate's pallet-contracts [22, 31] and our post-quantum ledger all execute WebAssembly [13], and Ethereum [30] and Solana [32] have their own virtual machines. None of the last five has STARK-curve host functions, so each verifier brings its own field and curve arithmetic. Our implementation techniques are classical: Montgomery multiplication [16], simultaneous (Straus) double-scalar multiplication [28], and batch inversion [17].

3 The signature scheme

Parameters. Let $p = 2^{251} + 17 \cdot 2^{192} + 1$ and let $\mathbb{G}$ be the STARK curve $E : y^2 = x^3 + x + \beta$ over $\mathbb{F}_p$. $\mathbb{G}$ has prime order q and cofactor 1, with generator G. Let $H : \mathbb{F}_p^* \to \mathbb{F}_p$ be Poseidon (the Hades permutation with Starknet's parameters) over field-element sequences, with ASCII domain tags `zuma-h2p`, `zuma-msg`, `zuma-chal`, `zuma-key`, `zuma-mine` and `zuma-score`. Challenges $c \in \mathbb{F}_p$ are used as scalars modulo q. Since $|p - q| < 2^{127}$, reducing a uniform element of $\mathbb{F}_p$ modulo q is within statistical distance 2^{-124} of uniform on $\mathbb{Z}_q$.

Hash to curve. $H_{\mathrm{p}}(\sigma)$ is defined by try-and-increment [5]. Take the least $k \geq 0$ for which $X_k = H(\texttt{zuma-h2p}, \sigma, k)$ is the x-coordinate of a point of E; then $H_{\mathrm{p}}(\sigma) = (X_k, \min(y, p - y))$. In the random-oracle model [1] this is a random point whose discrete logarithm to base G is unknown. A verifier may take y as a *hint*. It then checks $y^2 = X_k^3 + X_k + \beta$, checks that y is the smaller root,

and checks with Euler's criterion that every X_j, $j < k$, is a non-residue. A wrong hint therefore never changes the result; it only forfeits the cheaper path.

Keys and canonical rings. A secret is $x \in \mathbb{Z}_q^*$, its public key is $P = xG$, and its key identifier is $\mathrm{id}(P) = H(\texttt{zuma-key}, P_x, P_y)$. A ring $\mathcal{R} = (P_0, \ldots, P_{n-1})$ is *canonical* if its identifiers are strictly increasing. A verifier rejects every non-canonical ring, so the same key cannot appear twice to make a ring look larger than it is.

Signing. To sign $m \in \mathbb{F}_p$ in scope σ with secret x at position π of a canonical ring $\mathcal{R}$, the signer computes $I = x\,H_{\mathrm{p}}(\sigma)$ and

$$d = H(\texttt{zuma-msg}, \sigma, m, I, n, P_0, \ldots, P_{n-1}).$$

It picks $\alpha \leftarrow \mathbb{Z}_q$ and sets $c_{\pi+1} = H(\texttt{zuma-chal}, d, \alpha G, \alpha H_{\mathrm{p}}(\sigma))$. For $i = \pi+1, \ldots, \pi-1$ (indices modulo n) it picks $s_i \leftarrow \mathbb{Z}_q$ and sets

$$L_i = s_i G + c_i P_i, \quad R_i = s_i H_{\mathrm{p}}(\sigma) + c_i I, \quad c_{i+1} = H(\texttt{zuma-chal}, d, L_i, R_i),$$

and it closes the ring with $s_\pi = \alpha - c_\pi x \bmod q$. The signature is $(c_0, s_0, \ldots, s_{n-1}, I)$.

Verification. The verifier checks that $\mathcal{R}$ is canonical and that every P_i and I lie on E. It recomputes d, and then L_i, R_i and c_{i+1} for $i = 0, \ldots, n-1$ starting from c_0. It accepts if and only if $c_n = c_0$.

This is the scheme of Liu, Wei and Wong [15] with the group generator for key images replaced by the per-scope point $H_{\mathrm{p}}(\sigma)$, as in traceable ring signatures [10]. The message digest binds the scope, the message, the image, the ring size and the ring itself. We claim no new cryptography; what the following sections examine is the composition.

3.1 Properties of the scheme

We use the following properties. All are in the random-oracle model, with H and H_{p} as independent random oracles.

Definition 1 (Unforgeability with respect to insider corruption)**.** *No PPT adversary that may corrupt some ring members and obtain signatures from the others can output a valid signature on a fresh $(\mathcal{R}, \sigma, m)$ in which no member of $\mathcal{R}$ is corrupted, except with negligible probability.*

Definition 2 (Scope linkability)**.** *For any scope σ and any PPT adversary holding the secrets of k members of the universe of keys, the adversary cannot output $k+1$ valid signatures in scope σ with pairwise distinct key images, over rings drawn from that universe, except with negligible probability.*

Definition 3 (Non-frameability)**.** *No PPT adversary can output a valid signature in scope σ whose key image equals $x\,H_{\mathrm{p}}(\sigma)$ for the secret x of an uncorrupted member, except with negligible probability, even after seeing signatures by that member in other scopes and in σ itself.*

Definition 4 (Anonymity in a fresh scope)**.** *Let P_0, P_1 be honest keys in a ring $\mathcal{R}$. Suppose the adversary knows the secrets of every other member of $\mathcal{R}$ and may obtain signatures by P_0 and P_1 in any scope other than a challenge scope σ^*. The adversary is then given a signature in σ^* by P_b, for a uniform bit b. It must not guess b with advantage better than negligible.*

action	signer	scope σ	message	key image
submit	a miner	$H(\texttt{zuma-mine}, \mathfrak{s}, e)$	work commitment w	the miner's tag τ
score	a validator	$H(\texttt{zuma-score}, \mathfrak{s}, e, \tau)$	score $s \in \{0..100\}$	a nullifier ν
settle	anyone	–	(e, τ)	–
claim	a miner	$H(\texttt{zuma-mine}, \mathfrak{s}, e)$	recipient r	τ again

Table 1: The four actions. Here τ and ν denote the identifier $\mathrm{id}(I)$ of the key image. The contract checks the ring against the role's registry (*submit*, *claim*: $\mathcal{M}$; *score*: $\mathcal{V}$), checks $n_{\min} \le n \le n_{\max}$, and rejects a repeated image in the same scope.

Theorem 1. *In the random-oracle model, the scheme is unforgeable with respect to insider corruption, scope linkable and non-frameable if the discrete-logarithm problem is hard in $\mathbb{G}$. It is anonymous in a fresh scope if the decisional Diffie–Hellman (DDH) problem is hard in $\mathbb{G}$.*

Proof sketch. The proofs follow Liu, Wei and Wong [15]; we note where scopes change them. *Unforgeability and linkability.* By the forking lemma [23], an adversary that closes a ring can be rewound on the oracle query that fixes some c_{j+1}. Two accepting transcripts with $c_j \neq c'_j$ but equal (L_j, R_j) yield $x_j = (s_j - s'_j)/(c'_j - c_j)$ with $P_j = x_j G$ *and* $I = x_j H_{\mathrm{p}}(\sigma)$. That is, every accepting signature is bound to one ring member's secret, and that secret is consistent with its image. A forgery for an uncorrupted ring therefore yields a discrete logarithm. For linkability, $k+1$ distinct images in one scope with only k known secrets force some extracted x_j to belong to an uncorrupted key, or to be a second logarithm of a corrupted key, which is impossible. *Non-frameability.* An image $xH_{\mathrm{p}}(\sigma)$ of an honest key, attached to an accepting signature, extracts to x, and finding x is discrete logarithm. The honest key's own signatures in σ are simulated by programming H (honest-verifier zero knowledge of the underlying 1-of-n sigma protocol under Fiat–Shamir [9]). *Anonymity.* Given a DDH tuple $(G, X = xG, Y, Z)$, program $H_{\mathrm{p}}(\sigma^*) = Y$, use X as P_b's key and Z as the challenge image, and simulate the ring by programming H. If $Z = xY$ the view is a real signature by P_b. Otherwise the image is independent of both keys and b is information-theoretically hidden. Signatures in other scopes are simulated with independent H_{p} points of known logarithm $H_{\mathrm{p}}(\sigma) = r_\sigma G$, using $I_\sigma = r_\sigma X$. □

Corollary 2 (Cross-scope unlinkability)**.** *Under DDH in the random-oracle model, the images of one key in distinct scopes $\sigma \neq \sigma'$ are jointly indistinguishable from the images of two independent keys.*

4 The protocol

A subnet deployment has a *subnet element* $\mathfrak{s} \in \mathbb{F}_p$ that binds every scope to that one contract (Section 6 gives its definition per chain), an epoch length *tempo* in blocks, a minimum ring size $n_{\min}$ enforced on chain, and an emission E per epoch. The epoch of a block b is $e = \lfloor b/\mathit{tempo} \rfloor$. The contract keeps two append-only registries, $\mathcal{M}$ (miners) and $\mathcal{V}$ (validators). A key belongs to at most one of them.

Actions. *Submit* (by a miner, while epoch e is open) records the miner's tag τ with commitment w; a second submission with the same image is rejected. The epoch's first submit takes a snapshot $V_e = |\mathcal{V}|$. *Score* (by a validator, while e is open, on a tag of e) adds s to the tag's histogram $h_\tau[s]$ and spends ν; a second score with the same ν is rejected. *Settle* (by anyone, after e closes, once

per tag) runs Zuma consensus (Section 5) and stores (C_τ, R_τ). *Claim* (by a miner, after every tag of e is settled) verifies a signature in the miner's epoch scope on the recipient r, requires its image to be a tag of e that has not been claimed, and credits $\lfloor E \cdot R_\tau / \sum_{\tau'} R_{\tau'} \rfloor$ to r.

Relaying. A transaction's sender is public on every chain we target, so the anonymous actions never check the caller. A member hands its signed payload to any relayer, which posts it and pays the fee. Only registry administration (*register*, *set_open*) is restricted to the owner.

4.1 Security of the protocol

Proposition 3. *Assume the scheme satisfies Theorem 1. Then, except with negligible probability, the following hold.*

(P1) Each miner has at most one tag per epoch, and only registered miners have tags.

(P2) Each validator contributes at most one score to each tag, and only registered validators score.

(P3) A tag's credit is claimed at most once, and only by the holder of the secret that created the tag.

(P4) A claim credits the recipient its owner signed. A relayer cannot redirect it.

(P5) An observer of the chain, including validators, the relayer and the other members, learns nothing more about which member performed an action than that it is a member of the action's ring. In particular a miner's tags in different epochs, and a validator's nullifiers on different tags, are mutually unlinkable.

Proof. (P1) and (P2) are scope linkability in the mine scope and in each score scope respectively, together with unforgeability, since the ring is checked against the registry. (P3): the claim scope equals the submit scope, so by non-frameability only the creator's secret can reproduce the tag's image, and the stored claimed flag makes the claim single-use. (P4) is unforgeability: the recipient is the signed message, and the digest d binds it. (P5) is anonymity in a fresh scope for each action. Corollary 2 covers linking across scopes, because epochs give distinct mine scopes and tags give distinct score scopes. Within a scope, linking is by design: a submission and its claim share an image, which is what (P3) requires. □

Remark 1 (Cross-deployment replay). Every scope contains $\mathfrak{s}$, which is derived from the contract's own address. A signature produced for one deployment, on the same chain or another, is valid in no other deployment.

5 Zuma consensus

Without identities there is no stake per validator, and no bond can be carried from round to round. We therefore keep only the clipping step of Yuma consensus. A ring is a single stake tier: *membership is the stake*, and every validator weighs one.

Definition 5 (Zuma consensus)**.** *Fix an epoch e with snapshot $V = V_e$ and $\kappa = \frac{1}{2}$. For a tag τ with scores $s_1, \ldots, s_{n_\tau}$, where validators that did not score count as 0, let $V_\tau = \max(V, n_\tau)$ and $k = \lceil \kappa V_\tau \rceil$. Then*

$$C_\tau = \max\{\, s \in \{0, \ldots, 100\} : |\{i : s_i \geq s\}| \geq k \,\} \quad (0 \text{ if } n_\tau < k),$$
$$R_\tau = \textstyle\sum_i \min(s_i, C_\tau), \qquad \mathrm{pay}_\tau = \left\lfloor E \cdot R_\tau \,/\, \sum_{\tau'} R_{\tau'} \right\rfloor.$$

chain	verifier	curve and hash arithmetic	subnet element $\mathfrak{s}$
Starknet	Cairo contract	native EC and Poseidon builtins	contract address
NEAR	Rust → wasm (near-sdk 5)	9×29-bit limbs, Straus, hinted H_{p}	sha256(account) mod 2^{251}
Ethereum	Solidity + Yul	Yul field ops; Poseidon as a separate contract	contract address
Solana	sBPF program	verification split into <1.4 M-CU units	felt stored in the subnet account
Substrate	ink! 5 → wasm	NEAR's `fp/lsag/poseidon` sources, verbatim	blake2$_{256}$(AccountId) mod 2^{251}
postquant	pqc1 wasm (off chain)	NEAR's sources; ML-DSA-44 receipts	sha256(contract path) mod 2^{251}

Table 2: Verifiers. Each chain's native suite verifies a fixture signed by the Cairo prover and checks H_{p}, key identifiers and scopes against Cairo's values.

On-chain evaluation. *Score* maintains the histogram h_τ (dense or sparse) and the sum $S_\tau = \sum_i s_i$. *Settle* walks $s = 100, 99, \ldots$ and accumulates $a(s) = \sum_{u \geq s} h_\tau[u]$ and $m(s) = \sum_{u \geq s} u\, h_\tau[u]$ until $a(s) \geq k$. Then $C_\tau = s$ and $R_\tau = S_\tau - m(s) + s \cdot a(s)$. This takes at most 101 histogram reads per tag, independent of the number of validators. The sparse variant (NEAR, Substrate, postquant) visits only the distinct scores; randomised tests check it against the dense reference walk of `src/core/yuma.py`.

Proposition 4 (Sub-quorum coalitions cannot raise consensus)**.** *Let A be a coalition of $a < k$ validators that may score arbitrarily, and let $\hat{s}_1 \geq \hat{s}_2 \geq \cdots$ be the scores of the remaining validators, with non-voters counting as 0. Then $C_\tau \leq \hat{s}_{k-a}$. Moreover each coalition score contributes at most C_τ to R_τ, so the coalition's total contribution to R_τ is at most $a \cdot \hat{s}_{k-a}$.*

Proof. For any s, $|\{i : s_i \geq s\}| \leq a + |\{j : \hat{s}_j \geq s\}|$. If $C_\tau = s$, the left side is at least k, so at least $k - a$ honest scores are $\geq s$, that is, $\hat{s}_{k-a} \geq s$. The second claim follows from the clipping $\min(s_i, C_\tau)$. □

A validator that is secretly also a miner, and so knows its own tag, gains from inflating that tag only up to the $(k-a)$-th honest score. Inflating beyond the honest consensus requires a coalition of at least $k = \lceil V/2 \rceil$, an honest majority bound in the spirit of Yuma's κ-majority. Taking V at the epoch's first submit prevents validators registered later from shifting k. Counting non-voters as 0 prevents a small set of active validators from forming a quorum by itself.

What is not carried over. Yuma's validator bonds and dividends reward validators that agree with consensus *over time*. They need a validator identity that persists across tags and epochs, which the nullifiers deliberately destroy. Rewarding validators while keeping the nullifiers unlinkable needs a further mechanism, for instance a per-epoch validator claim over the score scopes that is itself unlinkable. We leave this to future work (Section 9).

6 One signer, six verifiers

There is exactly one signer. It is a Cairo program (`src/core/prover`) that links the same `lsag` module the Starknet contract runs, plus a line-by-line Python port (`src/core/lsag.py`) whose outputs are tested to be bit-identical to it. Every chain only *verifies*. A chain is one adapter package (`src/networks/<chain>/`) that implements a fixed interface of 19 methods. The registry discovers adapters automatically, and a conformance test runs against every adapter.

Making the STARK curve affordable without host support. Starknet executes the curve natively. Every other environment pays per instruction. On NEAR, the first port used a generic big-integer field and needed 214 Tgas to verify a ring of 4, which would have capped rings at about five members under the 300 Tgas call limit. Four changes brought this down to 36 Tgas. (i) A field representation shaped for p: nine 29-bit limbs, so that a column of nine 29×29-bit products fits in 64 bits without carries. Since $p \equiv 1 \pmod{2^{29}}$ and p has only three nonzero limbs in radix 2^{29}, Montgomery reduction [16] needs no multiplications. (ii) Explicit wrapping arithmetic, so that wasm compilers do not lower checked 64-bit multiplies to library calls. (iii) Jacobian coordinates with mixed addition and a 4-bit-window Straus double-scalar multiplication [28] for each L_i and R_i, with normalisation by batch inversion [17]. (iv) Poseidon on the same field, and the H_{p} hint of Section 3. The square root it replaces is expensive here because the 2-adicity of $p-1$ is 192. Every optimised path is differentially tested against a reference implementation, including the exceptional cases of the group law. These sources are the ones the Substrate and postquant verifiers reuse unchanged.

6.1 The Substrate verifier

Contract. `src/networks/substrate/contract` is an ink! 5.1.1 contract [22] for pallet-contracts. It has no copy of the cryptography. A build script reads NEAR's `fp.rs`, `lsag.rs` and `poseidon.rs` and adds the `alloc` imports that a `no_std` crate needs, so one source file implements the verifier on three platforms. Felts travel as 32-byte big-endian arrays and are rejected, never reduced, if they are $\geq p$. The subnet element is $\mathrm{blake2}_{256}$ of the contract's AccountId with the top five bits cleared; the claim message is the same function of the recipient's AccountId. Errors are a typed enum, and returning one reverts the call's storage changes. The Zuma histogram is sparse: one storage cell per tag, holding sorted (score, count) pairs.

Build. Two incompatibilities had to be resolved, and both are pinned in the repository. First, `lambdaworks-math` 0.13 (via `starknet-types-core`) depends on `getrandom` with its JavaScript backend for every `wasm32` target. That backend needs `std`, which a contract cannot link, even though the crate never calls it. We patch `getrandom` 0.2 with a short stand-in that fails inside the contract and forwards to `getrandom` 0.3 off-chain. Second, cargo-contract 5.0.3 passes a `build-std` feature that rustc removed in 1.91, while lambdaworks needs rustc $\geq$1.87, so we build with rustc 1.88. cargo-contract builds from a temporary copy of the manifest, which breaks the relative `[patch]` path; setting `TMPDIR` to the adapter directory makes the same relative path resolve from both places. The optimised wasm is 149 KB.

Wire and weight. The Python client encodes calls in SCALE itself, using selectors from the contract metadata. Every write is first dry-run through the `ContractsApi_call` runtime API, which returns the value, the revert flag and the weight consumed, and then submitted as `Contracts.call` with that weight plus 20% as its gas limit. Weight has two dimensions. *ref_time* is execution time in picoseconds against the benchmarked schedule. *proof_size* is the bytes of state proof the call would add to a parachain block.

Clock. The development node seals one block per extrinsic. To advance the epoch clock, the adapter seals empty `System.remark` blocks; a public network simply waits for blocks.

chain	native suite	passed	Zuma = reference	replayed claim	epoch (s)
Starknet	snforge (Cairo)	10/10	yes	already claimed	57.1
NEAR	cargo test (near-sdk)	32/32	yes	already claimed	71.8
Ethereum	forge (Foundry)	15/15	yes	already claimed	9.3
Solana	cargo test (sBPF)	13/13	yes	already claimed	123.9
Substrate	cargo test (ink! off-chain env)	13/13	yes	already claimed	15.4
postquant	cargo test (pqc1)	3/3	yes	reverted: already claimed	28.1

Table 3: Correctness. "Epoch" is the wall-clock time of a full local epoch. It reflects each local node's block production, not on-chain cost, and is not comparable across chains.

		submit		score		settle		claim	
chain	unit	cost	USD	cost	USD	cost	USD	cost	USD
Starknet	Cairo steps	15 979	0.0017	17 133	0.0019	13 862	0.0016	16 091	0.0017
NEAR	Tgas	29.6	0.014	38	0.018	2.61	0.0013	29.5	0.014
Ethereum	gas	2.29×10^6	3.45	2.34×10^6	3.52	174 537	0.262	2.27×10^6	3.41
Solana	CU (all units)	3.86×10^7	0.024	3.89×10^7	0.025	5 739	0.0011	3.85×10^7	0.024
Substrate	ms ref_time	28.7	0.0059	29.7	0.0061	1.38	0.0003	28.8	0.0059
postquant	ms wasm (off chain)	6.79	–	8.48	–	0.036	–	6.89	–

Table 4: Mean cost per action at ring size 4, from the last full epoch on each chain, priced on mainnet on 2026-10-20. Starknet USD is an estimate: Sierra-gas weights of steps and builtins, excluding the account's own validation. Solana's cost is summed over the units the verification is split into. postquant runs off chain and is not priced.

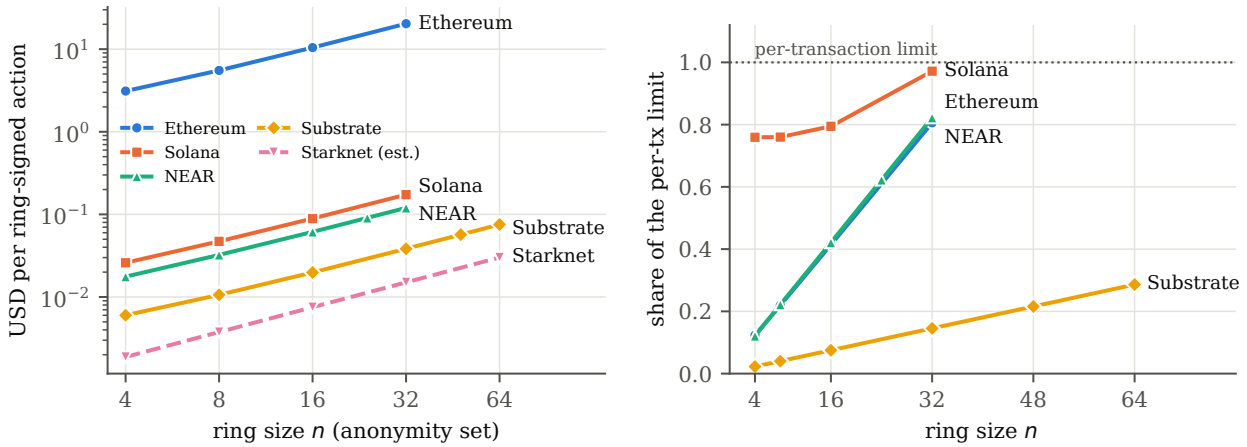
7 Evaluation

Setup. All measurements were made on one host (Intel Xeon Silver 4116, 24 threads) against each chain's local node: starknet-devnet 0.2.4 (Scarb 2.8.4), near-sandbox (neard 2.0.0), anvil 1.7.1 (Prague rules), solana-test-validator 4.3.2 (Agave), substrate-contracts-node 0.42.0, and the postquant node. The quantity measured is each chain's *own resource accounting*: Cairo steps and builtins, NEAR gas burnt, EVM gas used, Solana compute units, and Substrate weight. These are deterministic functions of the executed code and its inputs, not wall-clock timings. Repeated runs therefore vary only with the inputs, for example the score value and the position of an entry in the sparse histogram. Across the 12 scores of one epoch the ranges were: Starknet 16 957–17 576 steps; NEAR 36.9–39.2 Tgas; Ethereum 2.32–2.41 Mgas; Solana 38.5–39.3 MCU; Substrate 29.0–31.0 ms. Ring sweeps used fresh deployments and one *submit* per ring size. USD figures apply each mainnet's fee parameters and the CoinGecko price, read once on 2026-10-20 (`paper/data/snapshot.json`). They depend on that day's prices and congestion and should be read as orders of magnitude.

End-to-end correctness. On each local chain, the evaluation registers four miners and four validators, runs one epoch with three tags and twelve scores at ring size 4, settles every tag, claims one tag and replays the claim. The settled (C_τ, R_τ) are compared with the reference implementation. Table 3 shows that all six environments agree with the reference and reject the replay, and that all 86 native tests pass.

chain	unit	a (fixed)	b (per member)	R^2	$n_{\max}$	binding limit
Starknet	Cairo steps	17 133 (ring 4)	–	–	64	own choice
NEAR	Tgas	6.27	7.51	0.999998	32	300 Tgas per call
Ethereum	kgas	412.07	409.37	0.999990	32	2^{24} gas per tx (EIP-7825)
Solana	MCU	2.08	9.44	0.999993	32	1.4 M CU per tx (per unit)
Substrate	ms	6.41	5.71	0.999998	64	1.3 s ref_time per extrinsic

Table 5: Least-squares cost models $\text{cost}(n) = a + bn$ for one submit with ring size n. Sweeps: NEAR $n \in \{4, 8, 16, 24, 32\}$; Ethereum and Solana $\{4, 8, 16, 32\}$; Substrate $\{4, 8, 16, 32, 48, 64\}$. Solana splits verification into units of at most 1.4 M compute units each, so its total grows linearly while the binding quantity is the largest single unit.



(a) USD per ring-signed action (log–log). Starknet (dashed) is extrapolated linearly from its measured ring-4 cost.

(b) Share of the per-transaction limit used by one submit. For Solana this is the largest single verification unit.

Figure 1: Cost against anonymity-set size. Each series is labelled directly; the values are in Tables 5 and 6.

Cost models. Verification performs $2n$ double-scalar multiplications and n Poseidon calls, plus a constant number of table builds and a hash-to-curve. We therefore fit $\text{cost}(n) = a + b\,n$ by least squares to each ring sweep (Table 5). The fit is near-exact on every chain ($R^2 \geq 0.99999$). On Substrate a ring member costs 5.71 ms of reference time and 10.08 KB of proof size, over fixed costs of 6.4 ms and 234 KB. The per-member proof size reflects pallet-contracts' benchmarked worst-case charge for each registry read, not the bytes actually proven. Inverting the fits against each chain's per-transaction limit predicts the largest affordable ring. For NEAR it predicts $n \leq 39$; a ring of 40 was indeed rejected for exceeding the gas limit, while 32 used 82% of it (Fig. 1b). For Ethereum it predicts $n \leq 39$ under EIP-7825's 2^{24}-gas cap [8]. Substrate's 1.3 s per-extrinsic ref_time would allow about 226 members. We cap Substrate at 64, like Starknet, where a ring uses 29% of the limit, and the contract enforces the cap (a ring-72 submission is rejected with `RingAboveMax`).

Observations. (1) For every chain we fitted, cost is linear in the anonymity set (Fig. 1a). The privacy-cost trade-off is therefore explicit: doubling the anonymity set roughly doubles the fee. (2) Native curve support matters most on Ethereum. With no STARK-curve precompile and an L1 gas price, a ring-4 action costs dollars there, 600 to 1 900 times more than on Substrate or Starknet.

chain	token	block (s)	1 h tempo (blocks)	USD, $n = 4$	$n = 32$	$n = 64$	actions/block
Starknet	STRK	1.67	2 156	0.0019	0.015	0.030	–
NEAR	NEAR	0.614	5 864	0.018	0.120	–	26
Ethereum	ETH	12	300	3.11	20.33	–	26
Solana	SOL	0.218	16 518	0.026	0.173	–	2
Substrate	ASTR	6.3	572	0.0060	0.038	0.075	50

Table 6: Mainnet economics on 2026-10-20: USD per ring-signed action by ring size, the number of blocks in the one-hour minimum epoch that public deployments enforce, and how many ring-4 scores one block can carry under the chain's capacity limit. Substrate is priced on Astar, whose fee is linear in ref_time (327.4 ASTR per second of weight, times a fee multiplier of 0.1 on that day) plus a length fee; it does not charge for proof_size. Starknet's capacity is not limited by the actions we measured.

(3) The wasm environments differ mainly in metering and price. NEAR and Substrate execute the *same* verifier source. NEAR charges ≈ 7.5 Tgas per member against a 300 Tgas cap. Substrate charges ≈ 5.7 ms per member against 1.3 s: each member uses about a sixth as large a share of its limit (0.44% against 2.5%). (4) *Settle* is cheap everywhere, because it reads at most 101 histogram entries and verifies no signature.

8 Reducing cost without giving up privacy

Table 6 prices one action. A subnet pays for an epoch, and in an epoch the contract of Section 4 executes

$$\underbrace{M\,\mathrm{sub}(n)}_{\text{submit}} + \underbrace{V\,M\,\mathrm{sig}(n)}_{\text{score}} + \underbrace{M\,\mathrm{set}}_{\text{settle}} + \underbrace{M\,\mathrm{clm}(n)}_{\text{claim}}$$

with M miners, V validators and ring size n, where $\mathrm{sig}(n) = a + bn$ is the fitted model of Table 5. The score term grows with the *product* of the two sets, and each of its terms is a full ring verification. At the size of one Bittensor subnet ($M = 256$, $V = 64$, $n = 16$, one-hour epochs) it dominates the bill on every chain (first column of Table 7). Cost can be cut in three ways. Some cuts leak nothing, some trade security for cost, and some trade unlinkability for cost. We separate them because the obvious cuts are in the last group.

Free levers. (i) *Epoch length.* An epoch is the same set of transactions whatever its length, so the cost per day falls as 1/hours. Longer epochs reveal nothing new; they only delay payouts. (ii) *Ring size as a privacy floor.* Cost is linear in n (Fig. 1a). A deployment should therefore choose the smallest n it accepts as an anonymity set, set $n_{\min} = n$ on chain, and let the client's fixed partition (Section 9) give every member exactly that set. Members gain nothing by paying for a larger ring, since their linked actions must use the same block anyway. (iii) *Chain.* At equal privacy the same epoch costs 1 396 times more on the most expensive chain (Ethereum L1) than on the cheapest (Starknet) (Table 7). (iv) *Relay batching.* One relayer transaction can carry many members' payloads. That saves only the per-transaction base fee (21 000 gas against $\approx$2.3 Mgas on Ethereum, one signature fee on Solana), but it also mixes senders, so it improves privacy.

Cuts that link actions. The standard compressions all work by merging signatures. *Batch*: a validator signs its whole score vector once per epoch, as Bittensor's `set_weights` does, with one

nullifier per (epoch). That replaces VM verifications with V. *Lazy*: a miner claims K epochs with one signature. *Proof*: on top of both, every signature is checked off chain and the epoch lands as one validity proof. Each step breaks part of (P5). Batching publishes one validator's scores for every tag, under one nullifier, as a vector. The vector links those scores to each other, and its pattern can fingerprint the validator across epochs, which is exactly the persistent identity the nullifiers were chosen to destroy. A lazy claim links K of a miner's epoch tags, as claiming K times to one address would.

Cuts that keep (P5). Two changes keep every signature exactly as it is: one tag per miner per epoch, one nullifier per validator per tag.

Secret committees (sortition). A validator may score tag τ only if

$$H(\nu) < \frac{C}{V} \cdot p, \qquad \nu = \mathrm{id}\big(x \cdot H_{\mathrm{p}}(\sigma_{e,\tau})\big),$$

the nullifier it would publish anyway, with $H(\nu)$ read as an integer in $[0, p)$. Only the validator can compute ν, and the ring signature that publishes it also proves the selection, so the contract checks one extra comparison. The committee is unknown until its members act, anyone can verify it, and it reveals nothing beyond the nullifier, so (P5) holds unchanged. A key cannot be ground for a favourable committee, because ν is fixed by the key and the scope. To stop a coalition from grinding *miner* keys against the coalition's committees before registering them, the scope must include an epoch beacon fixed after the registries freeze for that epoch. A tag then draws about C scores instead of V, so the score term becomes $CM\,\mathrm{sig}(n)$.

Private settlement. Every submit, score and claim is verified off chain, and one validity proof per epoch states that all signatures verify against the registries, that their images are distinct in their scopes, and that Zuma consensus gives the published results. The images are scoped to the epoch, so their uniqueness can be checked entirely inside the proof; they never need to be stored. On chain remain one proof verification, M tags, M results and M claim records. The prover sees only what is public today: anonymous signatures. It is trusted for liveness, not for soundness. It can omit actions but not forge them, so the direct path of Section 4 stays open as a fallback, and anyone may submit a competing proof that includes more actions. Individual scores need not be published at all, only each tag's (C_τ, R_τ). That is *more* private than the baseline, which reveals each score's value. Claims are made once per epoch, each to a fresh recipient, so no tags are linked.

The price of committees is security, not privacy. Zuma's clipping bound (Proposition 4) now holds per committee. We keep the rule $V_\tau = \max(V, n_\tau)$ in committee form, with quorum $k_\tau = \lceil \max(C, n_\tau)/2 \rceil$. A coalition with fewer than $C/2$ members can then never reach the quorum. A larger one reaches it on a tag only when the draw favours it (Table 8). With $V = 64$, $C = 32$ and a quarter of the validators colluding, a tag is captured with probability 2.2×10^{-7} and starved with probability 0.0066. With $C = 16$ these become 0.0041 and 0.061. Starved tags can be re-scored by a second committee drawn in a retry scope; that costs another C scores, but only for starved tags. Committees save the most when $V \gg C$: for $V = 64$ and $C = 32$ they halve the score term, while for $V = 256$ they cut it by a factor of eight.

What the numbers say. Once verification moves off chain, ring size and signature count cost only prover time. At that point linking actions buys almost nothing: on every chain, +private costs the same order per day as +proof, and less on NEAR, Ethereum and Solana, where it also moves claims into the proof (Table 7), and it costs 307 to 5776 times less than today's contract.

		links actions			keeps (P5)		
chain	today	batch	+lazy	+proof	committee	+private	saving
Starknet	3052.81	260.13	219.10	8.55	1574.04	9.93	307×
NEAR	24952.88	995.51	624.83	20.90	12867.55	7.53	3313×
Ethereum	4261949.26	185464.82	117201.16	3887.86	2203365.91	1223.55	3483×
Solana	36074.28	1268.90	777.18	25.12	18553.49	6.25	5776×
Substrate*	8051.55	302.49	187.43	10.47	4146.63	7.13	1129×

Table 7: USD per day for a Bittensor-sized subnet ($M = 256$ miners, $V = 64$ validators, ring $n = 16$, one-hour epochs) at the prices of 2026-10-20. "Today" applies the measured per-action costs. The other columns are modeled (`src/core/forecast.py`): lazy claims $K = 24$ epochs at once, committees have expected size $C = 32$, a written entry is priced at a quarter of a measured settle, off-chain proving at USD 10^{-4} per million Cairo steps (about 4 000 steps per ring member), and the on-chain proof check as a Groth16 wrap (350 kgas on Ethereum, 300 kCU on Solana, 30 Tgas on NEAR, 5×10^7 L2 gas for a Garaga verifier on Starknet). *No proof-verifier estimate; priced at 30 ring-16 scores. "Saving" is today over +private.

		capture			starve		
V	C	$f = 0.10$	0.25	1/3	0.10	0.25	1/3
64	16	0	0.0041	0.039	0.012	0.061	0.12
64	32	0	$< 10^{-6}$	0.0011	1.5×10^{-4}	0.0066	0.033
64	48	0	0	0	$< 10^{-6}$	5.2×10^{-5}	0.0018
256	16	6.9×10^{-6}	0.0092	0.059	0.022	0.083	0.16
256	32	$< 10^{-6}$	3.0×10^{-4}	0.012	0.0023	0.026	0.083
256	48	$< 10^{-6}$	6.7×10^{-6}	0.0023	2.0×10^{-4}	0.0079	0.043

Table 8: The security price of committees, per tag. A coalition holds a fraction f of the V validators. Each validator is selected independently with probability C/V, and the committee quorum is $\lceil \max(C, n_\tau)/2 \rceil$. *Capture*: the coalition reaches the quorum while every honest member drawn votes, so it sets the tag's consensus. *Starve*: fewer than $\lceil C/2 \rceil$ honest members are drawn, so a coalition that abstains zeroes the tag. With $C \geq V$ both are 0 for $f < \frac{1}{2}$ (Proposition 4).

The cuts that link actions only pay off when no prover is available. We therefore recommend the second ladder: committees first, because they need only one comparison in the existing contracts, then private settlement. These figures are models of contract changes we have not deployed. The baseline column is the only measured one.

9 Limitations and threats to validity

The anonymity set is the ring, and rings are small. A score from a ring of 4 hides its validator among 4. Rings are canonical and their minimum size is enforced on chain, so a ring cannot be padded, but no contract can create members who do not exist. With a few validators the guarantee is weak, regardless of the cryptography.

Ring intersection: a leak in our earlier ring selection, and the fix. A claim intentionally shares its key image with its submission (P3). In earlier versions, the client sampled a *fresh random* ring for every action whenever the role had more members than the maximum ring. The

submission and the claim then show two different rings with the same image. Any observer can intersect them, and the claimant's effective anonymity set falls from n to the overlap: about $n^2/|\mathcal{M}|$ members on average, and one member in the worst case. This is the ring-intersection attack known from Monero [14, 18]. It did not affect the evaluation, where every set fit within one ring, but it would affect any deployment whose set exceeds $n_{\max}$. The client now partitions each registry, in registration order, into fixed blocks of n members; the last block borrows from the one before so that every block is full. A member always signs with its own block, so the two linked actions show the same n keys and intersection reveals nothing. A residual leak remains when registrations between a submission and its claim reshuffle the last, partial block. We test the partition rule; enforcing a canonical ring per (scope, member) *on chain* remains open.

Metadata. The relayer's identity, the time of posting, network addresses and the content of the work can each identify a member. A miner whose work is recognisable is not anonymous to the validators who read it. Relaying through independent relayers and mixing submission times are operational requirements, not features of the protocol.

Collusion and Sybils. Proposition 4 assumes that fewer than half of the snapshot validators collude. Anyone who controls admission to $\mathcal{V}$ can violate that assumption. Sybil resistance is out of scope [6].

No quantum security. Unforgeability rests on discrete logarithms and anonymity on DDH in $\mathbb{G}$, and Shor's algorithm [26] breaks both. A quantum adversary can forge signatures and can link key images to keys retroactively. The postquant deployment signs each state transition with ML-DSA [19]. That makes the *history* tamper-evident against such an adversary but does nothing for anonymity. Post-quantum linkable ring signatures from lattices exist [7], as do hash-based membership proofs from STARKs [3]; adopting one is the main open direction.

The cost reductions are modeled. Committees and private settlement (Section 8) are not deployed. Their costs rest on estimates of a written entry, of proving speed and of each chain's proof-verification cost. Committees weaken the deterministic sub-quorum bound to the probabilistic one of Table 8, and private settlement adds a prover that can delay an epoch, though it cannot forge one.

Validator rewards. Zuma consensus pays miners only (Section 5). Validator rewards that preserve nullifier unlinkability are future work.

Threats to validity. The costs are local measurements. Mainnet costs differ where fee markets add priority fees, or where account abstraction adds validation cost, as on Starknet. Starknet's USD figure is an estimate from Sierra-gas weights, and its by-ring curve is extrapolated from ring 4. Ethereum's figure uses the base fee observed on the snapshot date and moves linearly with it. The proofs in Section 4.1 are sketches in the random-oracle model, modelling Poseidon as a random oracle. That is a standard assumption, but not a reduction to Poseidon's concrete security. Finally, all six verifiers are tested against the same signer. A bug common to the signer and every verifier would not be caught by cross-chain agreement. It would have to survive the independent Cairo, Solidity and sBPF implementations as well as the shared Rust sources.

10 Conclusion

The rules of an incentive subnet can be enforced without the identities that currently enforce them. A single linkable ring signature, used in three carefully chosen scopes, gives one submission per miner per epoch, one score per validator per submission, and a claim only the earner can make. Zuma consensus keeps Yuma's majority clipping with anonymous, equally weighted validators. The cost is linear in the anonymity set and fits comfortably within transaction limits on Starknet and Substrate. On NEAR and Solana it is bounded at 32 members, and on Ethereum L1 it is expensive. At scale the cost is driven by miners times validators. It can be cut by orders of magnitude without linking any action: secret committees drawn from the nullifiers, then one validity proof per epoch. The cheaper-looking batched weights and lazy claims give up unlinkability for savings that vanish once a prover exists. The limits are equally concrete: small sets give small anonymity, metadata can betray members, the construction is not post-quantum, and validator rewards remain open.

A Reproducing the results

```
python3 -m pytest                            # offline: codecs, adapters, Zuma, ring rule
ZTENSOR_NATIVE=1 python3 -m pytest -m native   # snforge, cargo, forge suites
ZTENSOR_LIVE=1   python3 -m pytest -m live     # one full epoch per local chain
m ztensor/simulate chain=substrate         # epoch on substrate-contracts-node :51337
m ztensor/gas_sweep chain=substrate sizes=4,8,16,32,48,64
m ztensor/forecast chain=near miners=16,256 validators=4,64   # both ladders
m ztensor/experiment chain=near strategy=private
python3 paper/collect.py && python3 paper/figures.py && ./paper/build.sh
```

References

[1] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In *ACM Conference on Computer and Communications Security (CCS)*, pages 62–73, 1993.

[2] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *IEEE Symposium on Security and Privacy*, pages 459–474, 2014.

[3] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.

[4] Adam Bender, Jonathan Katz, and Ruggero Morselli. Ring signatures: Stronger definitions, and constructions without random oracles. In *Theory of Cryptography – TCC 2006*, volume 3876 of *LNCS*, pages 60–79. Springer, 2006.

[5] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 514–532. Springer, 2001.

[6] John R. Douceur. The Sybil attack. In *Peer-to-Peer Systems (IPTPS 2002)*, volume 2429 of *LNCS*, pages 251–260. Springer, 2002.

[7] Muhammed F. Esgin, Raymond K. Zhao, Ron Steinfeld, Joseph K. Liu, and Dongxi Liu. MatRiCT: Efficient, scalable and post-quantum blockchain confidential transactions protocol. In *ACM Conference on Computer and Communications Security (CCS)*, pages 567–584, 2019.

[8] Ethereum Improvement Proposals. EIP-7825: Transaction gas limit cap. `https://eips.ethereum.org/EIPS/eip-7825`, 2026. Accessed October 2026.

[9] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology – CRYPTO '86*, volume 263 of *LNCS*, pages 186–194. Springer, 1987.

[10] Eiichiro Fujisaki and Koutarou Suzuki. Traceable ring signature. In *Public Key Cryptography – PKC 2007*, volume 4450 of *LNCS*, pages 181–200. Springer, 2007.

[11] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a Turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Paper 2021/1063, 2021.

[12] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In *30th USENIX Security Symposium*, pages 519–535, 2021.

[13] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and J. F. Bastien. Bringing the web up to speed with WebAssembly. In *ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 185–200, 2017.

[14] Amrit Kumar, Clément Fischer, Shruti Tople, and Prateek Saxena. A traceability analysis of Monero's blockchain. In *Computer Security – ESORICS 2017*, volume 10493 of *LNCS*, pages 153–173. Springer, 2017.

[15] Joseph K. Liu, Victor K. Wei, and Duncan S. Wong. Linkable spontaneous anonymous group signature for ad hoc groups. In *Information Security and Privacy – ACISP 2004*, volume 3108 of *LNCS*, pages 325–335. Springer, 2004.

[16] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985.

[17] Peter L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of Computation*, 48(177):243–264, 1987.

[18] Malte Möser, Kyle Soska, Ethan Heilman, Kevin Lee, Henry Heffan, Shashvat Srivastava, Kyle Hogan, Jason Hennessey, Andrew Miller, Arvind Narayanan, and Nicolas Christin. An empirical analysis of traceability in the Monero blockchain. *Proceedings on Privacy Enhancing Technologies*, 2018(3):143–163, 2018.

[19] National Institute of Standards and Technology. Module-lattice-based digital signature standard. FIPS 204, 2024.

[20] Shen Noether. Ring signature confidential transactions for Monero. Cryptology ePrint Archive, Paper 2015/1098, 2015.

[21] Opentensor Foundation. Yuma consensus. Bittensor documentation, `https://docs.bittensor.com`, 2026. Accessed October 2026.

[22] Parity Technologies. ink! 5 and pallet-contracts: smart contracts for Substrate. `https://use.ink`, 2026. ink! 5.1.1; substrate-contracts-node 0.42.0. Accessed October 2026.

[23] David Pointcheval and Jacques Stern. Security arguments for digital signatures and blind signatures. *Journal of Cryptology*, 13(3):361–396, 2000.

[24] Yuma Rao, Jacob Steeves, Ala Shaabana, Daniel Attevelt, and Matthew McAteer. BitTensor: A peer-to-peer intelligence market. arXiv:2003.03917, 2020.

[25] Ronald L. Rivest, Adi Shamir, and Yael Tauman. How to leak a secret. In *Advances in Cryptology – ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 552–565. Springer, 2001.

[26] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *35th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 124–134, 1994.

[27] Alex Skidanov and Illia Polosukhin. Nightshade: Near protocol sharding design. Whitepaper, 2019.

[28] Ernst G. Straus. Addition chains of vectors (problem 5125). *American Mathematical Monthly*, 71:806–808, 1964.

[29] Nicolas van Saberhagen. CryptoNote v 2.0. Whitepaper, 2013.

[30] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Yellow Paper, 2014.

[31] Gavin Wood. Polkadot: Vision for a heterogeneous multi-chain framework. Whitepaper, 2016.

[32] Anatoly Yakovenko. Solana: A new architecture for a high performance blockchain. Whitepaper v0.8.13, 2018.